Practical: 11
Date: 23/10/2026

Great Feature Engineering

Dataset Description: The Ames Housing dataset contains residential home sale prices for Ames, Iowa, along with various explanatory variables.

Procedure:
Step 1: Load the dataset using pandas.
Step 2: Inspect missing values and data types.
Step 3: Perform necessary feature engineering operations such as filling missing values and transforming categorical variables.
Step 4: Train a regression model such as XG Boost or Random Forest.
Step 5: Evaluate the model performance and interpret the results

Solution:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# Step 1: Load the dataset
train = pd.read_csv("C:/Users/SDM/Downloads/dataset/train.csv")
test = pd.read_csv("C:/Users/SDM/Downloads/dataset/test.csv")

print("Training Dataset:")
print(train.head())
```

```
print("Training Shape:", train.shape)
print("Testing Shape:", test.shape)

# Step 2: Inspect missing values
print("\nDataset Information:")
train.info()

missing = train.isnull().sum()
print("\nMissing Values:")
print(missing[missing > 0].sort_values(ascending=False))

# Step 3: Feature engineering and preparation
train = train.dropna(subset=["SalePrice"])

test_ids = test["Id"]

X = train.drop(columns=["SalePrice", "Id"])
y = train["SalePrice"]
X_test_final = test.drop(columns=["Id"])

# Convert MSSubClass into a categorical variable
X["MSSubClass"] = X["MSSubClass"].astype(str)
X_test_final["MSSubClass"] = X_test_final["MSSubClass"].astype(str)

# Identify numerical and categorical columns
num_cols = X.select_dtypes(include="number").columns.tolist()
cat_cols = [col for col in X.columns if col not in num_cols]

# Handle missing numerical values
num_pipe = Pipeline([
  ("imputer", SimpleImputer(strategy="median"))])

# Handle missing categorical values and encode categories
cat_pipe = Pipeline([
  ("imputer", SimpleImputer(strategy="constant", fill_value="None")),
  ("encoder", OneHotEncoder(handle_unknown="ignore"))])

preprocessor = ColumnTransformer([
  ("num", num_pipe, num_cols),
  ("cat", cat_pipe, cat_cols)])
```

```
# Step 4: Split the data
X_train, X_val, y_train, y_val = train_test_split(
   X, y, test_size=0.2, random_state=42)

# Create the Random Forest Regression model
model = Pipeline([
   ("preprocessor", preprocessor),
   ("regressor", RandomForestRegressor(
      n_estimators=300,
      random_state=42,
      n_jobs=-1))])

# Train the model
model.fit(X_train, y_train)

# Predict validation prices
y_pred = model.predict(X_val)

# Step 5: Evaluate model performance
mae = mean_absolute_error(y_val, y_pred)
mse = mean_squared_error(y_val, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_val, y_pred)

print("\nModel Evaluation Results")
print("MAE:", round(mae, 2))
print("MSE:", round(mse, 2))
print("RMSE:", round(rmse, 2))
print("R2 Score:", round(r2, 4))
print("R2 Percentage:", round(r2 * 100, 2), "%")

# Compare actual and predicted prices
results = pd.DataFrame({
   "Actual Price": y_val.values,
   "Predicted Price": y_pred})

print("\nActual and Predicted Prices:")
print(results.head(10))
```

```
# Step 6: Feature importance
features = model.named_steps["preprocessor"].get_feature_names_out()
importance = model.named_steps["regressor"].feature_importances_

feature_table = pd.DataFrame({
    "Feature": features,
    "Importance": importance
}).sort_values(by="Importance", ascending=False)

print("\nTop 10 Important Features:")
print(feature_table.head(10).to_string(index=False))

# Step 7: Refit model on all training data
model.fit(X, y)

# Predict prices for the test dataset
test_pred = model.predict(X_test_final)

# Save predictions to a CSV file
submission = pd.DataFrame({
    "Id": test_ids,
    "SalePrice": test_pred})

submission.to_csv(
    "C:/Users/SDM/Downloads/submission.csv",
    index=False)

print("\nSubmission file saved successfully.")
print("Submission Shape:", submission.shape)

# Step 8: Graph 1 - Actual vs Predicted Prices
plt.figure(figsize=(8, 6))
plt.scatter(y_val, y_pred, alpha=0.7)

min_price = min(y_val.min(), y_pred.min())
max_price = max(y_val.max(), y_pred.max())

plt.plot(
    [min_price, max_price],
    [min_price, max_price],
```

```
    linestyle="--")

plt.xlabel("Actual House Prices")
plt.ylabel("Predicted House Prices")
plt.title("Actual vs Predicted House Prices")
plt.tight_layout()
plt.savefig("C:/Users/SDM/Downloads/graph1_actual_vs_predicted.png")
plt.show()

# Step 9: Graph 2 - Distribution of Prediction Errors
errors = y_val.values - y_pred

plt.figure(figsize=(8, 6))
plt.hist(errors, bins=30, edgecolor="black")
plt.xlabel("Prediction Error")
plt.ylabel("Frequency")
plt.title("Distribution of Prediction Errors")
plt.tight_layout()
plt.savefig("C:/Users/SDM/Downloads/graph2_prediction_errors.png")
plt.show()

# Step 10: Graph 3 - Top 10 Feature Importance
top_features = feature_table.head(10).sort_values(
    by="Importance", ascending=True)

plt.figure(figsize=(9, 6))
plt.barh(
    top_features["Feature"],
    top_features["Importance"])

plt.xlabel("Feature Importance")
plt.ylabel("Features")
plt.title("Top 10 Important Features")
plt.tight_layout()
plt.savefig("C:/Users/SDM/Downloads/graph3_feature_importance.png")
plt.show()

# Step 11: Graph 4 - Actual vs Predicted Prices for First 20 Houses
comparison = results.head(20)
positions = np.arange(len(comparison))
```

```
plt.figure(figsize=(12, 6))
plt.plot(positions,comparison["Actual Price"], marker="o",label="Actual Price")
plt.plot(positions,comparison["Predicted Price"],marker="x",label="Predicted Price")

plt.xlabel("House Number")
plt.ylabel("Sale Price")
plt.title("Actual vs Predicted Prices for First 20 Houses")
plt.legend()
plt.tight_layout()
plt.show()
```

Output:

Training Shape: (1460, 81)

Testing Shape: (1459, 80)

Model Evaluation Results

MAE: 17434.29

MSE: 827980520.53

RMSE: 28774.65

R2 Score: 0.8921

R2 Percentage: 89.21 %

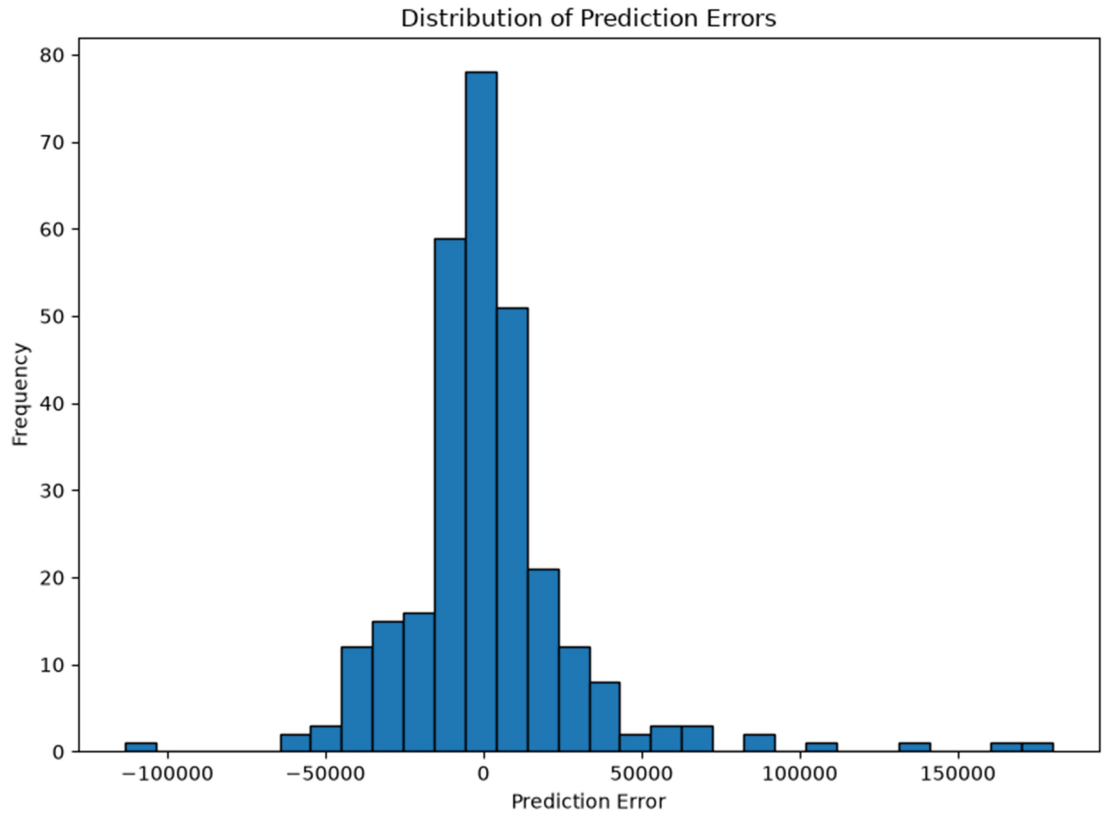
Actual and Predicted Prices:

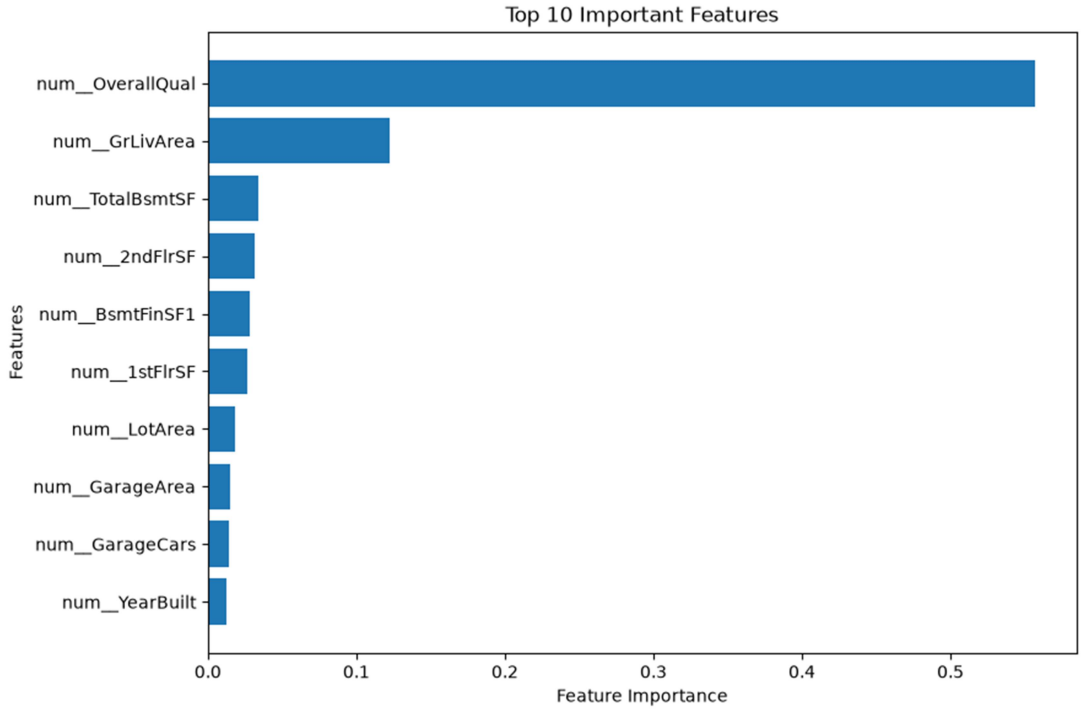
	Actual Price	Predicted Price
0	154500	140805.373333
1	325000	323361.286667

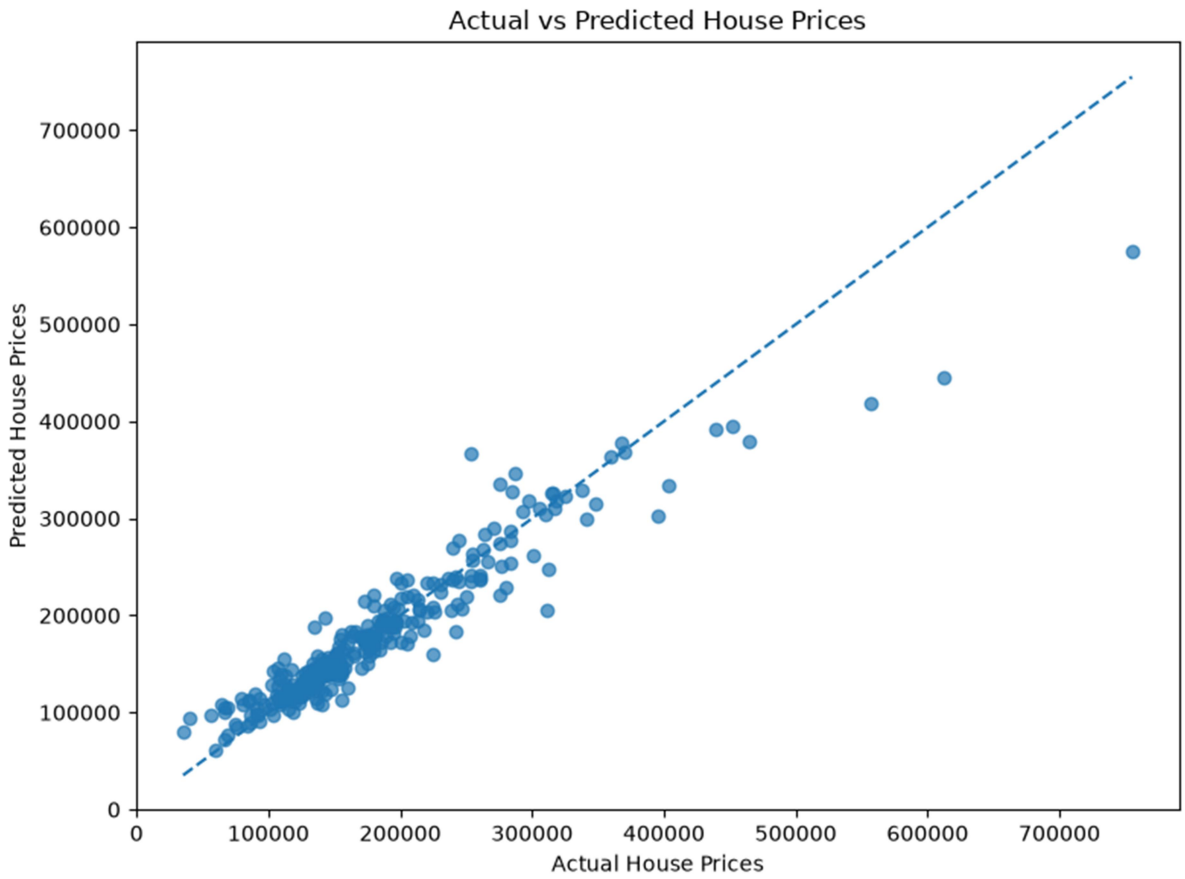
2	115000	116620.583333
3	159000	153983.516667
4	315500	325875.300000
5	75500	85259.833333
6	311500	205787.370000
7	146000	152700.573333
8	84500	85848.216667
9	135500	129553.043333

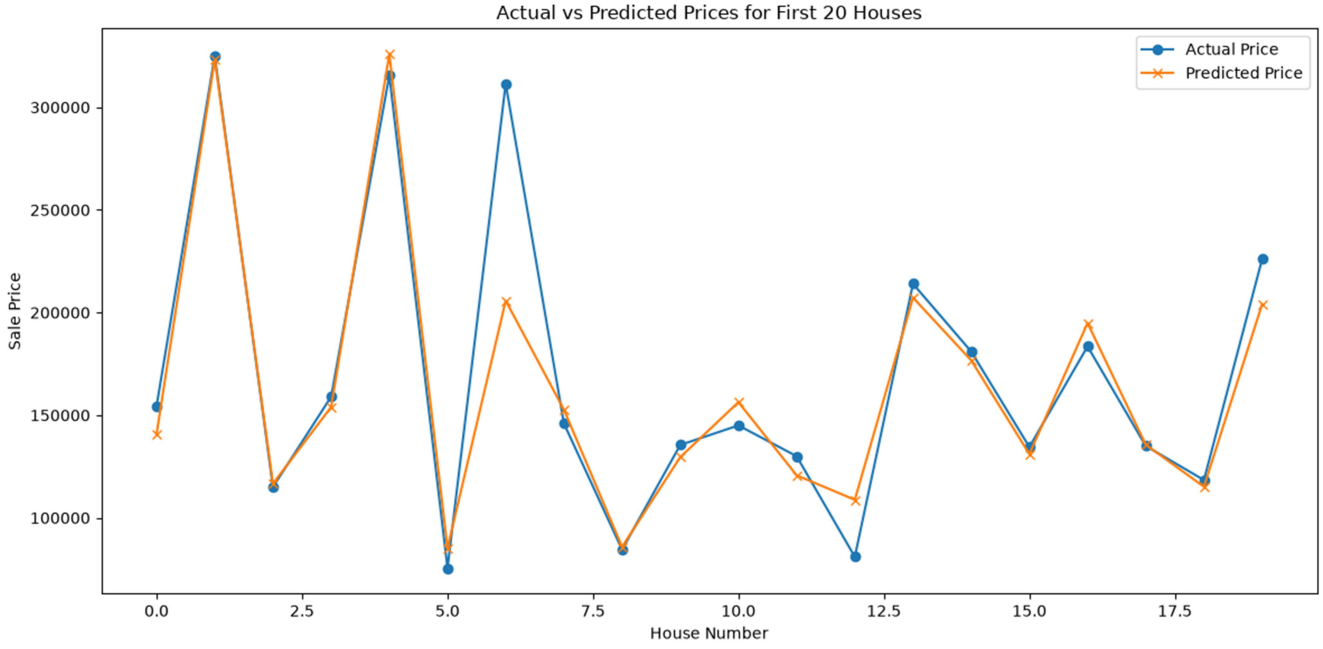
Top 10 Important Features:

Feature	Importance
num__OverallQual	0.557172
num__GrLivArea	0.122183
num__TotalBsmtSF	0.034070
num__2ndFlrSF	0.031701
num__BsmtFinSF1	0.028564
num__1stFlrSF	0.026192
num__LotArea	0.018594
num__GarageArea	0.015380
num__GarageCars	0.013911
num__YearBuilt	0.012381


Distribution of Prediction Errors
Frequency
Prediction Error
0
10
20
30
40
50
60
70
80
−100000
−50000
0
50000
100000
150000


Top 10 Important Features
Features
Feature Importance
num__OverallQual
num__GrLivArea
num__TotalBsmtSF
num__2ndFlrSF
num__BsmtFinSF1
num__1stFlrSF
num__LotArea
num__GarageArea
num__GarageCars
num__YearBuilt
0.0
0.1
0.2
0.3
0.4
0.5


Actual vs Predicted House Prices
Predicted House Prices
Actual House Prices
0
100000
200000
300000
400000
500000
600000
700000


Actual vs Predicted Prices for First 20 Houses
Actual Price
Predicted Price
Sale Price
House Number
100000
150000
200000
250000
300000
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5